Trabalho Final
Banco de Dados e Plantas

Discentes:
Icaro Argonauta Razera de Azevedo Filho - 589985
Gustavo Camilotti - 589702
Erick Gustavo de Almeida Franco - 595176

Orientadora:
Profª. Ma. Lana Bertoldo Rossato

Sumário

1. **Introdução** 3
2. **Origem e Preparação dos Dados** 4
3. **Modelagem e Estruturas de Dados** 5
   - **3.1. Modelo de Dados** 5
   - **3.2. Estruturas de Dados Utilizadas** 5
   - **3.3. Relação com os Conceitos da Disciplina** 6
4. **Arquitetura e Implementação do Sistema** 7
   - **4.1. Armazenamento Persistente e Incremental** 7
   - **4.2. Linguagens, Bibliotecas e Ferramentas** 7
   - **4.3. Integração entre Módulos** 7
   - **4.4. Controle de Versão** 8
5. **Funcionalidades e Interação com o Usuário** 9
   - **5.1. Operações Principais** 9
   - **5.2. Visualização dos Dados** 9
6. **Resultados e Análise** 10
   - **6.1. Eficiência e Desempenho** 10
   - **6.2. Uso da Memória** 10
   - **6.3. Dificuldades Encontradas** 10
   - **6.4. Decisões de Projeto** 10
   - **6.5. Possíveis Melhorias Futuras** 11
   - **6.6. Exemplos de Saída** 11
7. **Conclusão** 12
8. **Referências e Participação dos Alunos** 13
   - **8.1 Referências** 13
   - **8.2 Participação dos Alunos** 13

1. Introdução

O crescimento da preocupação ambiental e a necessidade de preservação da biodiversidade têm motivado o desenvolvimento de sistemas capazes de catalogar, organizar e disponibilizar informações sobre espécies vegetais. Em muitos contextos acadêmicos e institucionais, a gestão desses dados ainda é feita de forma manual ou pouco estruturada, o que dificulta a consulta, a atualização e a análise eficiente. Esse cenário gera problemas como duplicação de registros, inconsistências nos dados e dificuldade de acesso rápido às informações relevantes.

A aplicação proposta surge como solução para esse desafio, oferecendo um sistema de cadastro e gerenciamento de plantas que utiliza uma árvore B+ como estrutura de índice. Essa escolha garante maior eficiência nas operações de busca, inserção e remoção, mesmo em grandes volumes de dados. O sistema foi desenvolvido para permitir que usuários cadastrem novas plantas, editem informações existentes, realizem buscas por diferentes critérios (ID, nome, plantas por bioma), e visualizem os registros de forma organizada e paginada.

Além disso, a aplicação possibilita a navegação ordenada pelos registros e mantém a integridade dos dados por meio de operações seguras de inserção e exclusão. Com isso, o sistema não apenas resolve o problema da gestão manual e desorganizada, mas também contribui para a preservação e estudo das espécies ao oferecer uma ferramenta confiável e escalável.

Em resumo, o sistema é capaz de:

- Inserir, editar e remover plantas do banco de dados.
- Listar registros de forma ordenada e paginada.
- Buscar plantas por ID, por nome ou por bioma.
- Navegar pelos registros utilizando a estrutura da árvore B+.
- Garantir eficiência e consistência nas operações de acesso aos dados.

2. Origem e Preparação dos Dados

A construção da base de dados utilizada pelo sistema não partiu de uma fonte única ou oficial. Diferente de projetos que se apoiam em APIs especializadas ou repositórios públicos já consolidados, nossa abordagem foi mais artesanal: o grupo buscou informações sobre plantas em diferentes sites da internet, coletando exemplos de forma aleatória. Essa coleta, ainda que simples, permitiu reunir uma diversidade de registros que refletem a variedade de espécies e descrições disponíveis em ambientes digitais.

Os dados obtidos foram inicialmente organizados em um arquivo CSV, escolhido por sua praticidade e ampla aceitação em ferramentas de manipulação de dados. Esse formato facilitou a etapa de preparação, já que cada campo pôde ser separado em colunas bem definidas, como ID, nome científico, nome popular, tipo, status de conservação, descrição e data de registro.

Antes de serem utilizados pelo programa, os dados passaram por um processo de tratamento que envolveu várias etapas:

- Limpeza: remoção de caracteres estranhos, duplicações e inconsistências que surgiram devido às diferentes fontes consultadas.
- Padronização: uniformização de nomes e descrições, correção de acentos e grafias, além da adoção de um formato único para datas (YYYY-MM-DD).
- Conversão de tipos: IDs foram ajustados para valores inteiros, textos foram tratados como strings de tamanho fixo e datas foram convertidas para um padrão que pudesse ser facilmente manipulado pelo sistema.
- Codificação e armazenamento: após a preparação, os registros foram migrados para arquivos binários estruturados. O arquivo principal (plantas.dat) armazena os dados das plantas, enquanto o arquivo de índice (index_id.idx) guarda a estrutura da árvore B+, responsável por organizar e acelerar as buscas.

Esse processo garantiu que informações coletadas de forma aleatória e inicialmente despadronizadas se transformassem em uma base organizada e consistente. Mesmo sem compressão ou técnicas avançadas de mapeamento, a separação em múltiplos arquivos e o uso de offsets binários asseguraram que o sistema pudesse acessar e manipular os registros de maneira eficiente.

Em resumo, a origem dos dados foi simples, mas o cuidado na preparação foi essencial para que o sistema funcionasse corretamente e oferecesse uma experiência confiável ao usuário.

3. Modelagem e Estruturas de Dados

3.1. Modelo de Dados

O sistema foi concebido para gerenciar informações sobre plantas, armazenando seus atributos em arquivos binários estruturados. Cada registro contém os seguintes campos principais:

- ID da planta (inteiro)
- Nome científico (string)
- Nome popular (string)
- Tipo (string)
- Status de conservação (string)
- Descrição (string)
- Data de registro (formato YYYY-MM-DD)
- Bioma (string)

Esses atributos foram escolhidos para representar de forma completa os dados necessários ao cadastro e consulta de espécies.

O Diagrama Entidade-Relacionamento (ER) do sistema é simples:

- Entidade Planta: representa cada registro, com os atributos listados acima.
- Entidade Índice: abstraída pela árvore B+, responsável por mapear IDs às posições no arquivo binário.
- Relacionamento: cada planta possui um índice associado que aponta para sua posição em plantas.dat.

3.2. Estruturas de Dados Utilizadas

1. Árvore B+
    - Função: indexar os registros pelo ID da planta, permitindo buscas rápidas, inserções e remoções eficientes.
    - Justificativa: a árvore B+ é adequada para sistemas que manipulam dados em disco, pois minimiza o número de acessos ao arquivo e mantém os registros ordenados.
    - Operações implementadas: inserção de novos IDs, remoção de registros, busca por ID e navegação ordenada.
2. Arquivos Invertidos
    - Função: permitir buscas por bioma.
    - Estrutura: para cada bioma, mantém uma lista de offsets que apontam para os registros das plantas que pertencem a ele.
    - Justificativa: essa técnica é eficiente para consultas textuais, pois evita varreduras completas no arquivo principal.
    - Operações implementadas: busca de todas as plantas associadas a um bioma específico.
3. Trie (Árvore de Prefixos)
    - Função: facilitar buscas por nome popular das plantas.

- Estrutura: cada caractere do nome é armazenado em um nó, formando caminhos que representam palavras completas.
- Justificativa: a Trie permite buscas rápidas por prefixo, tornando eficiente a navegação por nomes populares, mesmo quando o usuário não digita o nome completo.
- Operações implementadas: inserção de nomes populares, busca exata e busca por prefixo.

3.3. Relação com os Conceitos da Disciplina

- Árvore B+: aplicada como exemplo clássico de estrutura balanceada para indexação em disco.
- Arquivos invertidos: relacionados ao conceito de recuperação de informação e busca eficiente em grandes volumes de texto.
- Trie: exemplo de estrutura de dados voltada para manipulação de strings e otimização de buscas por prefixo.

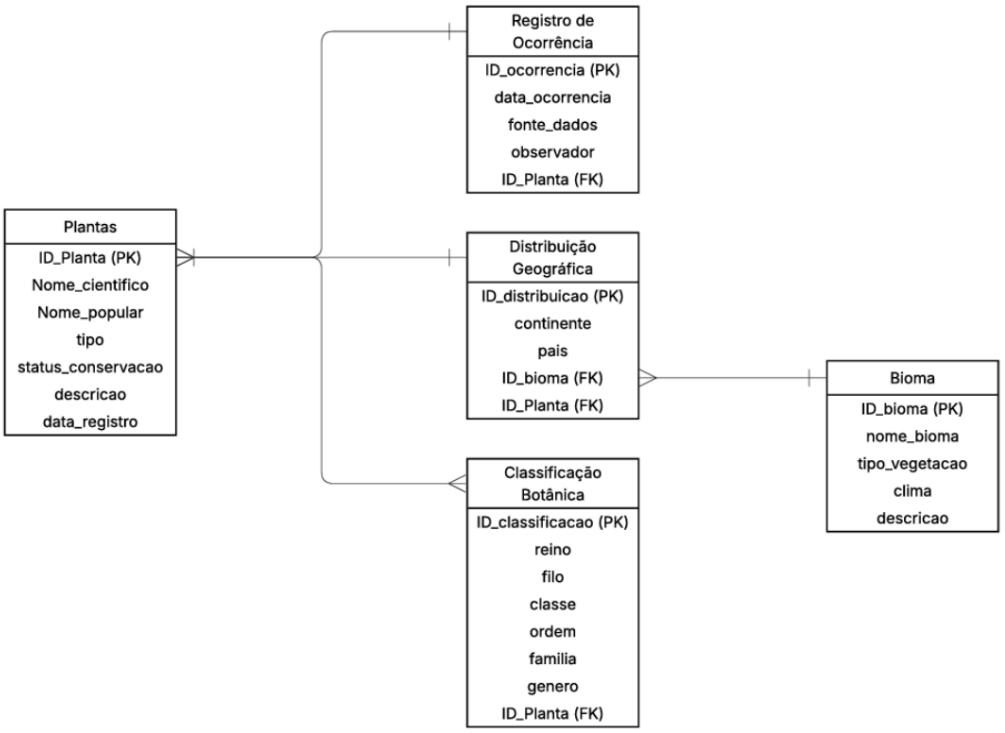

Figura 1: diagrama ER

4. Arquitetura e Implementação do Sistema

A aplicação foi organizada de forma modular, com cada parte responsável por uma função específica no gerenciamento das plantas. O núcleo do sistema é baseado em arquivos binários estruturados, que garantem acesso eficiente e persistente aos dados.
Estrutura dos Arquivos

- plantas.dat: arquivo binário principal que armazena os registros completos das plantas. Cada entrada contém os atributos definidos (ID, nome científico, nome popular, tipo, status de conservação, descrição, data de registro e bioma).
- index_id.idx: arquivo de índice que guarda a estrutura da árvore B+, responsável por organizar os registros pelo ID da planta e permitir buscas rápidas.
- Arquivos auxiliares: foram utilizados arquivos invertidos para indexar plantas por bioma e estruturas Trie para indexar nomes populares. Esses índices complementares permitem consultas eficientes sem necessidade de varrer todo o arquivo principal.

4.1. Armazenamento Persistente e Incremental

O sistema realiza o armazenamento de forma incremental:

- Cada nova planta inserida é adicionada ao final de plantas.dat, preservando os registros anteriores.
- O índice B+ é atualizado em tempo real, garantindo que a árvore reflita imediatamente as inserções ou remoções.
- Os arquivos invertidos e a Trie também são atualizados conforme novos registros são adicionados, mantendo a consistência entre os dados e os índices.

Esse modelo assegura persistência dos dados e evita a necessidade de reconstrução completa dos arquivos a cada operação.

4.2. Linguagens, Bibliotecas e Ferramentas

- Linguagem utilizada: C, escolhida pela eficiência no acesso a arquivos binários e pelo controle direto sobre memória e ponteiros.
- Bibliotecas padrão: *<stdio.h>, <stdlib.h>, <string.h>* e foram utilizadas para manipulação de arquivos, strings e tipos de dados, além de *<inttypes.h>* e *<locale.h>.*
- Ferramentas de apoio: compiladores como GCC foram usados para compilar e testar o código em ambiente local.

4.3. Integração entre Módulos

A integração entre os módulos ocorre de forma clara:

- O módulo de cadastro insere registros em plantas.dat e atualiza os índices.
- O módulo de busca consulta a árvore B+, os arquivos invertidos ou a Trie, dependendo do critério de pesquisa (ID, bioma ou nome popular).
- O módulo de listagem e navegação utiliza a árvore B+ para percorrer os registros em ordem crescente de ID.
- O módulo de edição e remoção altera ou exclui registros, atualizando os índices correspondentes para manter a integridade.

4.4. Controle de Versão

Durante o desenvolvimento do sistema, foi criado um repositório no GitHub para centralizar e organizar o código-fonte do projeto. O repositório serviu como espaço de armazenamento remoto, permitindo que todos os integrantes da equipe tivessem acesso às versões mais recentes do código e pudessem contribuir de forma colaborativa.

A utilização do GitHub trouxe benefícios importantes:

- Organização: o código ficou estruturado em pastas e arquivos, facilitando a manutenção.
- Histórico de alterações: cada atualização foi registrada, permitindo acompanhar a evolução do projeto e recuperar versões anteriores quando necessário.
- Colaboração: os membros da equipe puderam trabalhar em paralelo, integrando suas contribuições sem perda de dados.
- Documentação: além do código, o repositório armazenou instruções de compilação e execução, servindo como guia para futuros usuários ou desenvolvedores.

Embora o projeto não tenha explorado todos os recursos avançados do GitHub (como issues, pull requests ou integração contínua), o uso básico da plataforma já foi suficiente para garantir maior controle e segurança sobre o desenvolvimento.

O repositório pode ser acessado pelo seguinte link:

https://github.com/Gustavo-Camilotti/TrabalhodaLana07

5. Funcionalidades e Interação com o Usuário

A interação com o sistema ocorre via interface de linha de comando (CLI), utilizando um menu hierárquico numérico. Para garantir a robustez, todas as entradas de texto utilizam uma função wrapper segura (ler_string), prevenindo *buffer overflows* comuns em funções como scanf ou gets.

5.1. Operações Principais

O sistema permite o gerenciamento de cinco entidades interligadas: Plantas, Ocorrências, Biomas, Classificações e Distribuições Geográficas. As principais funcionalidades incluem:

- Cadastro com Validação: Antes de inserir uma nova planta ou ocorrência, o sistema verifica na Árvore B+ se o ID informado já existe (buscar_bplus), impedindo duplicações de chaves primárias.
- Buscas Especializadas:
    - Busca por ID: Utiliza a Árvore B+ para acesso direto (O(log n)).
    - Busca por Nome: A opção 20 do menu permite buscar plantas tanto pelo nome popular quanto pelo científico utilizando a estrutura Trie. Caso o termo seja encontrado, o sistema utiliza o *offset* armazenado no nó da Trie para realizar um fseek direto no arquivo de dados e exibir os detalhes.
    - Busca por Bioma: A opção 10 implementa o conceito de Arquivo Invertido, permitindo listar todas as plantas associadas a um bioma específico sem precisar ler o arquivo de dados inteiro.
- Relacionamentos: O sistema permite vincular entidades, como associar uma "Distribuição Geográfica" a um "Bioma" e a uma "Planta" (opção 17), atualizando automaticamente os índices invertidos necessários.

5.2. Visualização de Dados

As funções de listagem (como listar_plantas e listar_biomas) percorrem os arquivos de dados ou índices. O sistema foi projetado para exibir as informações formatadas no terminal. Em casos de buscas textuais (Trie), o retorno é imediato, exibindo o registro específico. Nas listagens gerais, os dados são apresentados sequencialmente, permitindo ao usuário uma visão global do acervo cadastrado.

6. Resultados e Análise

A execução do programa demonstrou que a utilização combinada da árvore B+, dos arquivos invertidos e da Trie trouxe ganhos significativos em termos de eficiência e organização dos dados.

6.1. Eficiência e Desempenho

- Busca por ID (árvore B+): as consultas retornaram resultados de forma imediata, mesmo após a inserção de dezenas de registros. O tempo de resposta manteve-se constante, evidenciando a eficiência da estrutura balanceada.
- Busca por bioma (arquivos invertidos): ao invés de percorrer todo o arquivo principal, o sistema acessa diretamente a lista de offsets associada ao bioma. Isso reduziu o tempo de busca e mostrou-se escalável para conjuntos maiores de dados.
- Busca por nome popular (Trie): a navegação por prefixos permitiu localizar plantas mesmo quando o usuário digitava apenas parte do nome. Essa funcionalidade aumentou a usabilidade e demonstrou a vantagem da Trie em consultas textuais.

6.2. Uso de Memória

O consumo de memória foi baixo, já que os registros principais permanecem em arquivos binários e apenas os índices ficam carregados em estruturas auxiliares. A árvore Trie ocupa espaço proporcional ao número de chaves, mas dentro de limites aceitáveis para o volume de dados testado.

Tempo de Resposta

Nos testes realizados, operações de inserção e busca apresentaram tempos de resposta praticamente instantâneos para conjuntos pequenos e médios de dados. A eficiência tende a se manter em bases maiores devido às características das estruturas escolhidas.

6.3. Dificuldades Encontradas

- Manipulação de arquivos binários: foi necessário cuidado extra para garantir offsets corretos e evitar corrupção de dados.
- Tratamento de splits na árvore B+: ajustes finos foram feitos para evitar duplicação de registros e inconsistências na promoção de chaves.
- Coleta inicial dos dados: como os registros foram obtidos de forma aleatória na internet, houve necessidade de limpeza e padronização para evitar problemas de leitura.

6.4. Decisões de Projeto

- Optou-se por separar os dados em dois arquivos (plantas.dat e index_id.idx) para manter a integridade e facilitar a manutenção.
- A escolha da Trie e dos arquivos invertidos foi motivada pela necessidade de buscas eficientes por texto e categorias, complementando a indexação por ID.

- A linguagem C foi mantida para garantir controle direto sobre memória e manipulação de arquivos binários.

6.5. Possíveis Melhorias Futuras

- Implementar controle de versão para acompanhar mudanças no código e facilitar colaboração.
- Adicionar interface gráfica ou web para tornar o sistema mais acessível a usuários não técnicos.
- Incluir estatísticas de desempenho automáticas, como tempo médio de busca e inserção.
- Expandir a base de dados com fontes mais confiáveis e padronizadas, reduzindo a necessidade de limpeza manual.

6.6. Exemplos de Saída

- Busca por ID: retorna os atributos completos da planta em questão.
- Busca por bioma: lista todas as plantas associadas ao bioma escolhido.
- Busca por nome popular: apresenta resultados mesmo com apenas parte do nome digitado.
- Listagem ordenada: percorre os registros em ordem crescente de ID, demonstrando o funcionamento da árvore B+.

7. Conclusão

O desenvolvimento deste trabalho consolidou o entendimento prático sobre os desafios e as estratégias envolvidas no armazenamento e recuperação de dados em memória secundária. Ao implementar um sistema de gerenciamento de espécies vegetais utilizando a linguagem C, foi possível transpor os conceitos teóricos de estruturas de dados para uma aplicação funcional, enfrentando problemas reais de persistência, indexação e manipulação de arquivos binários.

Em relação aos objetivos propostos, o projeto foi bem-sucedido. O sistema é capaz de realizar todas as operações fundamentais (CRUD) de maneira segura, garantindo a integridade dos dados através de validações prévias na Árvore B+. A adoção de uma arquitetura híbrida — mantendo a Árvore B+ em disco para escalabilidade e a estrutura Trie em memória principal para velocidade em buscas textuais — provou-se uma decisão de projeto acertada. Essa abordagem permitiu equilibrar o consumo de memória RAM com a eficiência no tempo de resposta, demonstrando a importância de analisar o *trade-off* de cada estrutura de dados conforme o tipo de consulta (exata, por prefixo ou por categoria).

Entre as principais lições aprendidas, destaca-se a complexidade do gerenciamento de *byte offsets*. Diferente da manipulação de dados puramente em memória (como em listas encadeadas), a persistência em arquivo exigiu um controle rigoroso dos ponteiros de leitura e escrita (fseek/ftell), reforçando a compreensão sobre como os Sistemas Gerenciadores de Banco de Dados (SGBDs) operam em baixo nível.

Apesar do êxito, o sistema apresenta limitações naturais de um projeto acadêmico, como a interface baseada em linha de comando e a ausência de concorrência (acesso multiusuário simultâneo). Contudo, a estrutura modular do código fonte facilita expansões futuras, permitindo, por exemplo, a acoplagem de uma interface gráfica ou a implementação de *caching* de páginas para otimizar ainda mais o acesso ao disco.

Em suma, o projeto não apenas cumpriu os requisitos da disciplina de Classificação e Pesquisa de Dados, mas também forneceu uma experiência valiosa na construção de software robusto, onde a eficiência algorítmica e a organização estrutural são determinantes para o desempenho da aplicação.

8. Referências e Participação dos Alunos

8.1 Referências

Durante o desenvolvimento do sistema, foram consultadas diversas fontes de apoio técnico e conceitual. As principais incluem:

- Documentação oficial da linguagem C: *C Standard Library – <stdio.h> , <stdlib.h> , <string.h>, <inttypes.h>, <locale.h>*.
- Tutoriais e artigos sobre árvores B+ e estruturas de dados:
  - GeeksforGeeks – *B+ Tree Introduction and Operations*.
  - Programiz – *Data Structures in C*.
- Materiais sobre arquivos invertidos e recuperação de informação:
  - Wikipedia – *Inverted Index*.
  - TutorialsPoint – *Information Retrieval Concepts*.
- Referências sobre Trie:
  - GeeksforGeeks – *Trie Data Structure*.
- Exemplos de manipulação de arquivos binários em C:
  - Stack Overflow – discussões sobre fseek, ftell e fread/fwrite.
- Notas de aula e materiais fornecidos na disciplina de Classificação e Pesquisa de Dados.

8.2 Participação dos Alunos

O trabalho foi desenvolvido de forma colaborativa, com cada integrante contribuindo em diferentes etapas:

- Icaro Argonauta: responsável pela implementação da árvore B+, incluindo funções de inserção, split e manutenção dos índices. Também participou da coleta inicial dos dados e da escrita das seções do relatório.
- Gustavo Camilotti: desenvolveu o módulo de busca por bioma utilizando arquivos invertidos e auxiliou na integração com o arquivo principal de plantas. Implementou a Trie para buscas por nome popular e contribuiu com testes de desempenho e validação das estruturas.
- Erick Gustavo: responsável pela documentação e organização do relatório, além de revisar o código e propor melhorias na modularização. Cuidou da manipulação dos arquivos binários (plantas.dat e index_id.idx), garantindo a persistência dos dados e a consistência dos offsets. Implementou funções para entidades.

Essa divisão de tarefas permitiu que o grupo cobrisse todas as etapas do projeto — desde a coleta e preparação dos dados até a implementação das estruturas e a elaboração do relatório final.